
Modul 2: System Design və Production Architecture

Gün 31-60 - scaling, reliability, deployment, security və real case study-lər

Bu paket backend developer üçün server əsasları, sistem dizaynı, arxitektura, dizayn pattern-lər, performans, etibarlılıq və production düşüncəsini ardıcıl öyrənmək üçün hazırlanıb. Hər gün bir mövzu oxu, həmin günün praktik tapşırığını yaz, sonra öz sistem dizayn qeydlərini yenilə.

Təvsiyə olunan gündəlik ritm: 60-90 dəqiqə nəzəriyyə, 60-120 dəqiqə praktika, 20 dəqiqə review.

Modulun istifadə qaydası

1. Hər günün əvvəlində məqsədi oxu və həmin mövzunu öz sözlərlə izah etməyə çalış.
2. Nəzəriyyəni oxuduqdan sonra praktikanı mütləq kodla və ya dizayn sənədi ilə tamamla.
3. Mini-lab hissəsində ölçmə, test və ya failure scenario yarat. Production düşüncəsi burada formalaşır.
4. Müsahibə suallarına cavabı səsli izah et. Cavabın strukturunu sadə saxla: problem, seçim, trade-off, risk.

Gün 31: System design interview framework

Öyrənmə məqsədi

Günün sonunda 'System design interview framework' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. System design interview framework real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 32: Requirements, constraints və estimation

Öyrənmə məqsədi

Günün sonunda 'Requirements, constraints və estimation' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Requirements, constraints və estimation real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 33: Load balancing və horizontal scaling

Öyrənmə məqsədi

Günün sonunda 'Load balancing və horizontal scaling' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Load balancing və horizontal scaling real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 34: Replication, failover və read/write split

Öyrənmə məqsədi

Günün sonunda 'Replication, failover və read/write split' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Replication, failover və read/write split real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 35: Sharding və partitioning strategiyaları

Öyrənmə məqsədi

Günün sonunda 'Sharding və partitioning strategiyaları' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Sharding və partitioning strategiyaları real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 36: Consistency modelləri və CAP/PACELC

Öyrənmə məqsədi

Günün sonunda 'Consistency modelləri və CAP/PACELC' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Distributed sistemlərdə əsas çətinlik bir neçə node-un eyni həqiqəti eyni anda görməməsidir. Network partition, clock drift, retry duplication və partial failure normal hadisələrdir. Consistency, availability və latency arasında trade-off qaçılmazdır; sistemin biznes invariant-larına görə seçim edilməlidir.

Praktiki tətbiq

Strong consistency tələb edən və eventual consistency qəbul edən əməliyyatları ayır. Məsələn payment capture və notification send eyni consistency tələb etmir.

Mini-lab

Retry zamanı duplicate order yaranmaması üçün idempotency table dizayn et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Consistency modelləri və CAP/PACELC real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 37: Consensus və leader election mental modeli

Öyrənmə məqsədi

Günün sonunda 'Consensus və leader election mental modeli' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Distributed sistemlərdə əsas çətinlik bir neçə node-un eyni həqiqəti eyni anda görməməsidir. Network partition, clock drift, retry duplication və partial failure normal hadisələrdir. Consistency, availability və latency arasında trade-off qaçılmazdır; sistemin biznes invariant-larına görə seçim edilməlidir.

Praktiki tətbiq

Strong consistency tələb edən və eventual consistency qəbul edən əməliyyatları ayır. Məsələn payment capture və notification send eyni consistency tələb etmir.

Mini-lab

Retry zamanı duplicate order yaranmaması üçün idempotency table dizayn et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Consensus və leader election mental modeli real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 38: Distributed cache və CDN dizaynı

Öyrənmə məqsədi

Günün sonunda 'Distributed cache və CDN dizaynı' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Distributed cache və CDN dizaynı real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 39: File storage, object storage və media pipeline

Öyrənmə məqsədi

Günün sonunda 'File storage, object storage və media pipeline' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. File storage, object storage və media pipeline real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 40: Search systems və indexing pipeline

Öyrənmə məqsədi

Günün sonunda 'Search systems və indexing pipeline' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Search systems və indexing pipeline real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 41: Observability: logs, metrics, traces

Öyrənmə məqsədi

Günün sonunda 'Observability: logs, metrics, traces' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Reliability sistemin səhv olmaması deyil; səhv olanda idarə olunan davranış göstərməsidir. Timeout-suz network call production riskidir. Retry jitter-siz olarsa overload-u artırabilir. Circuit breaker, bulkhead, fallback və graceful degradation istifadəçi təsirini azaldır.

Praktiki tətbiq

External API çağırışı üçün timeout, retry with exponential backoff, circuit breaker və fallback planı yaz.

Mini-lab

Süni olaraq external API-ni yavaşlat və sistemin p95 latency, error rate və fallback davranışını ölç.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Observability: logs, metrics, traces real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 42: Timeout, retry, jitter və circuit breaker

Öyrənmə məqsədi

Günün sonunda 'Timeout, retry, jitter və circuit breaker' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Reliability sistemin səhv olmaması deyil; səhv olanda idarə olunan davranış göstərməsidir. Timeout-suz network call production riskidir. Retry jitter-siz olarsa overload-u artırabilir. Circuit breaker, bulkhead, fallback və graceful degradation istifadəçi təsirini azaldır.

Praktiki tətbiq

External API çağırışı üçün timeout, retry with exponential backoff, circuit breaker və fallback planı yaz.

Mini-lab

Süni olaraq external API-ni yavaşlat və sistemin p95 latency, error rate və fallback davranışını ölç.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Timeout, retry, jitter və circuit breaker real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 43: Graceful degradation və bulkhead pattern

Öyrənmə məqsədi

Günün sonunda 'Graceful degradation və bulkhead pattern' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Reliability sistemin səhv olmaması deyil; səhv olanda idarə olunan davranış göstərməsidir. Timeout-suz network call production riskidir. Retry jitter-siz olarsa overload-u artırabilir. Circuit breaker, bulkhead, fallback və graceful degradation istifadəçi təsirini azaldır.

Praktiki tətbiq

External API çağırışı üçün timeout, retry with exponential backoff, circuit breaker və fallback planı yaz.

Mini-lab

Süni olaraq external API-ni yavaşlat və sistemin p95 latency, error rate və fallback davranışını ölç.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Graceful degradation və bulkhead pattern real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 44: Deployment strategiyaları: blue-green, canary, rolling

Öyrənmə məqsədi

Günün sonunda 'Deployment strategiyaları: blue-green, canary, rolling' mövzusunun ham izah etməli, ham da kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Operations düşüncəsi kodun deploydan sonra necə yaşayacağını anlamaqdır. Config, secrets, logs, metrics, health checks, release strategy və rollback planı backend-in bir hissəsidir. Yaxşı deploy strategiyası istifadəçiyə təsiri minimuma endirir və komandaya sürətli bərpa imkanı verir.

Praktiki tətbiq

Servis üçün Dockerfile, healthcheck və environment config siyahısı yaz. Staging və production fərqlərini ayır.

Mini-lab

Canary deployment ssenarisi qur: 5 faiz traffic yeni versiyaya getsin, error rate artarsa rollback qərarı ver.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Deployment strategiyaları: blue-green, canary, rolling real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 45: Container, Docker image və runtime config

Öyrənmə məqsədi

Günün sonunda 'Container, Docker image və runtime config' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Operations düşüncəsi kodun deploydan sonra necə yaşayacağını anlamaqdır. Config, secrets, logs, metrics, health checks, release strategy və rollback planı backend-in bir hissəsidir. Yaxşı deploy strategiyası istifadəçiyə təsiri minimuma endirir və komandaya sürətli bərpa imkanı verir.

Praktiki tətbiq

Servis üçün Dockerfile, healthcheck və environment config siyahısı yaz. Staging və production fərqlərini ayır.

Mini-lab

Canary deployment ssenarisi qur: 5 faiz traffic yeni versiyaya getsin, error rate artarsa rollback qərarı ver.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Container, Docker image və runtime config real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 46: Kubernetes mental modeli

Öyrənmə məqsədi

Günün sonunda 'Kubernetes mental modeli' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Operations düşüncəsi kodun deploydan sonra necə yaşayacağını anlamaqdır. Config, secrets, logs, metrics, health checks, release strategy və rollback planı backend-in bir hissəsidir. Yaxşı deploy strategiyası istifadəçiyə təsiri minimuma endirir və komandaya sürətli bərpa imkanı verir.

Praktiki tətbiq

Servis üçün Dockerfile, healthcheck və environment config siyahısı yaz. Staging və production fərqlərini ayır.

Mini-lab

Canary deployment ssenarisi qur: 5 faiz traffic yeni versiyaya getsin, error rate artarsa rollback qərarı ver.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Kubernetes mental modeli real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 47: Secrets, config və environment management

Öyrənmə məqsədi

Günün sonunda 'Secrets, config və environment management' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Security backend-də sonradan əlavə olunan qat deyil; hər request üçün identity, permission, input trust və data exposure sualları verilməlidir. Authentication kim olduğunu, authorization nə edə biləcəyini müəyyən edir. Secrets, audit log, rate limit və threat modeling production sistemlərində minimum gözləntidir.

Praktiki tətbiq

Endpoint-ləri public, authenticated və owner-only qruplarına böl. Hər endpoint üçün kim, hansı resource-a, hansı şərtlə access alır cədvəlini yaz.

Mini-lab

JWT və session əsaslı login-in trade-off-larını müqayisə et. Token expiry, refresh, logout və compromised token senarilərini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Secrets, config və environment management real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 48: Security hardening və threat modeling

Öyrənmə məqsədi

Günün sonunda 'Security hardening və threat modeling' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Security backend-də sonradan əlavə olunan qat deyil; hər request üçün identity, permission, input trust və data exposure sualları verilməlidir. Authentication kim olduğunu, authorization nə edə biləcəyini müəyyən edir. Secrets, audit log, rate limit və threat modeling production sistemlərində minimum gözləntidir.

Praktiki tətbiq

Endpoint-ləri public, authenticated və owner-only qruplarına böl. Hər endpoint üçün kim, hansı resource-a, hansı şərtlə access alır cədvəlini yaz.

Mini-lab

JWT və session əsaslı login-in trade-off-larını müqayisə et. Token expiry, refresh, logout və compromised token senarilərini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Security hardening və threat modeling real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 49: Payment sistemlərinin riskləri

Öyrənmə məqsədi

Günün sonunda 'Payment sistemlərinin riskləri' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Payment sistemlərinin riskləri real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 50: Notification sistemləri: email, SMS, push

Öyrənmə məqsədi

Günün sonunda 'Notification sistemləri: email, SMS, push' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

System design təkcə qutu və ox çəkmək deyil; requirements, constraints, data model, API, capacity, storage, scaling, failure modes və trade-off-ları ardıcıl düşünməkdir. Güclü cavab 'nə seçdim' deyil, 'niyə bunu seçdim və nəyi qurban verdim' sualına cavab verir.

Praktiki tətbiq

Hər dizayn üçün bu şablonu doldur: functional requirements, non-functional requirements, traffic estimate, data size, API, data model, high-level architecture, bottlenecks, failure modes.

Mini-lab

URL shortener üçün 100M link və 10K RPS fərziyyəsi ilə storage və QPS hesablaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Notification sistemləri: email, SMS, push real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 51: URL shortener dizaynı

Öyrənmə məqsədi

Günün sonunda 'URL shortener dizaynı' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. URL shortener dizaynı real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 52: Chat/messaging sistemi dizaynı

Öyrənmə məqsədi

Günün sonunda 'Chat/messaging sistemi dizaynı' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Chat/messaging sistemi dizaynı real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 53: News feed və timeline sistemi

Öyrənmə məqsədi

Günün sonunda 'News feed və timeline sistemi' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. News feed və timeline sistemi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 54: Booking/reservation sistemi

Öyrənmə məqsədi

Günün sonunda 'Booking/reservation sistemi' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Booking/reservation sistemi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 55: E-commerce checkout sistemi

Öyrənmə məqsədi

Günün sonunda 'E-commerce checkout sistemi' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. E-commerce checkout sistemi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 56: Analytics event ingestion sistemi

Öyrənmə məqsədi

Günün sonunda 'Analytics event ingestion sistemi' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Analytics event ingestion sistemi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 57: Feature flag sistemi

Öyrənmə məqsədi

Günün sonunda 'Feature flag sistemi' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Feature flag sistemi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 58: Multi-tenant SaaS arxitekturası

Öyrənmə məqsədi

Günün sonunda 'Multi-tenant SaaS arxitekturası' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Multi-tenant SaaS arxitekturası real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 59: Data privacy, audit log və compliance

Öyrənmə məqsədi

Günün sonunda 'Data privacy, audit log və compliance' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Security backend-də sonradan əlavə olunan qat deyil; hər request üçün identity, permission, input trust və data exposure sualları verilməlidir. Authentication kim olduğunu, authorization nə edə biləcəyini müəyyən edir. Secrets, audit log, rate limit və threat modeling production sistemlərində minimum gözləntidir.

Praktiki tətbiq

Endpoint-ləri public, authenticated və owner-only qruplarına böl. Hər endpoint üçün kim, hansı resource-a, hansı şərtlə access alır cədvəlini yaz.

Mini-lab

JWT və session əsaslı login-in trade-off-larını müqayisə et. Token expiry, refresh, logout və compromised token senarilərini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Data privacy, audit log və compliance real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 60: Module 2 capstone: scalable system design review

Öyrənmə məqsədi

Günün sonunda 'Module 2 capstone: scalable system design review' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Module 2 capstone: scalable system design review real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.